CSE331
AUTOMATA AND COMPUTABILITY

Turing machines: encoding, proving decidability, proving undecidability

Spring 2026

Today's learning goals Sipser 3.1,3.2

- State and use the Church-Turing thesis. - Explain what it means for a problem to be decidable. - Justify the use of encoding.
- Give examples of decidable problems.
- Use counting arguments to prove the existence of unrecognizable (undecidable) languages. - Use diagonalization in a proof of undecidability.

Review

Is M a decider? ((M) = L(Mij n

L (Ma)

What's L(M)?

Consider the TM M = "On input w,

1. Run M_1 on w. If M_1 rejects, rejects. If M_1 accepts, go to 2. 2. Run M_2 on w. If M_2 accepts, accept. If M_2 rejects, reject."

What kind of construction is this?

A. Formal definition of TM
B. Implementation-level description of TM
C. High-level description of TM
D. I don't know.

Describing TMs

Sipser Section 3.3, p. 184

- **Formal definition**: set of states, input alphabet, tape alphabet, transition function, state state, accept state, M reject state.
- **Implementation-level definition**: English prose to describe Turing machine head movements relative to contents of tape.

#

#*

WHW

L

- **High-level description**: Description of algorithm, without [redacted] implementation details of machine. As part of this description, can "call" and run another TM as a [redacted] subroutine.

[redacted] M and M2 as helper machine, inside its

Subroutines

Computation

It is called a subroutine because M is using &wa

Consider the TM M = "On input w,

1. Run M_1 on w. If M_1 rejects, rejects. If M_1 accepts, go to 2. 2. Run M_2 on w. If M_2 accepts, accept. If M_2 rejects, reject."

-> procedure

main program :(m)

Call Function1)) (M,)

Call function 2()(Mz)

High-level description = Algorithm

- Wikipedia "self-contained step-by-step set of operations to be performed"
- "An algorithm is a finite sequence of precise instructions for performing a computation or for solving a problem."

Each algorithm can be implemented by

Church-Turing thesis
some Turing

machine. Calculus+M

At start of CSE 331…

- Pick a model of computation
- Study what problems it can solve
- Prove its limits


string
S
DFA
Input
TM
Complex input
String
(encoding)

Classification: is input of type A or not?

Decision problem

{ w | w is of type A}

PRIME = { 2, 3, 5, 7, … }
SORTED = { <1,3>, <-1, 8, 17> …}

Decision problems are coded by sets of strings

Encoding input for TMs *Sipser p. 159*

- By definition, TM inputs are **strings**
- To define

TM M: "On input w …

For inputs that aren't strings, we have to **encode the object** (represent it as a string) first

Encoding inputs

(BW) meansencoded String

1. .. 2. .. 3. …

↓

But sometimes we want to give in something more complex as <O_1, …, O_n> is the single string that

input, Such as : DFA , DDA, another TM , a graph t encode them as strings

w is a string

Notation:

<O> is the **string** that represents (encodes) the object O

represents the tuple of objects O_1, …, O_n

containing oa

Payoff: problems we care about can be reframed as languages of strings

wEL(B)

w = abba W ← DFAB - Yes WR = abba

w^R }

e.g. "Recognize whether a string is a This lang Contains all encoded pairs where palindrome." { w | w in {0,1}* and w =

= g DFA accepts given

e.g. "Check whether a string is String accepted by a DFA."

{ <B,w> | B is a DFA over Σ, w in Σ*, and w is in L(B) } e.g.

"Check whether the language of a PDA is infinite." { <A> | A is a PDA and L(A) is infinite}

Example: encoding a DFA

- Think of encoding as a text file that encodes your data (in some format), that then you store in binary

Example: encoding a DFA

- Think of encoding as a text file that encodes your data

(in some format), that then you store in binary

Example: encode a DFA=(Q,∑,δ,q_0,F) using

JSO

N -

Text format for representing data

{
“states”: [1,2,3,4,5], // Q={1,2,3,4,5} “alphabet”:
[0,1], // ∑={0,1}
“transition”: [((1,0),2), (1,1), 3), …], // $\delta(1,0) = 2, \delta(1,1) = 3,$

... “initial_state”: 1, // q_0=1
“accept_states”: [4,5], // F={4,5}
}

Computational problems

A computational problem is **decidable** iff the language encoding the problem instances is decidable

A Computational problem asks a yes/no question

For example,

Problem : Does DFA B accept string w ?

- yes

$< B, w] = TM^{M}$ - No

Computational problems

Sample computational problems and their encodings: • A_{DFA} "Check whether a string is accepted by a DFA." { <B,w> | B is a DFA over Σ, w in Σ*, and w is in L(B) } GUESaby*W starts with ay

• E_{DFA} "Check whether the language of a DFA is empty."

{ <A> | A is a DFA over Σ, L(A) is empty }

- EQ_{DFA} "Check whether the languages of two DFAs are equal."

$$=$$

$$\{ \langle A, B \rangle \mid A \text{ and } B \text{ are DFAs over } \Sigma, L(A) = L(B) \}$$

Regular lang = countable in finite

FACT: all of these problems are decidable!

Proving decidability

Claim: A_{DFA} is decidable

Proof: WTS that $\{ \langle B,w \rangle \mid B \text{ is a DFA over } \Sigma, w \text{ in } \Sigma^*, \text{ and } w \text{ is in } L(B) \}$ is decidable.

Step 1: construction

How would you check if w is in L(B)?

Proving decidability

Define TM M_1 by: M_1 = "On input <B,w>

1. Check whether B is a valid encoding of a DFA and w is a valid input for B. If not, reject.
2. Simulate running B on w (by keeping track of states in B, transition function of B, etc.)
3. When the simulation ends, by finishing to process all of w, check current state of B: if it is final, *accept*; if it is not, reject."

WEB WKB

Proving decidability

Step 1: construction -> decided

Define TM M_1 by M_1 = "On input <B,w>

1. Check whether B is a valid encoding of a DFA and w is a valid input for B. If not, reject.
2. Simulate running B on w (by keeping track of states in B, transition function of B, etc.)
3. When the simulation ends, by finishing to process all of w, check current state of B: if it is final, *accept*; if it is not, reject."

Step 2: correctness proof

WTS (1) $L(M_1) = A_{DFA}$ and (2) M_1 is a decider.

Proving decidability

Claim: E_{DFA} is decidable

=[21*

Ga-z ’0-9]

↑

Proof: WTS that { <A> | A is a DFA over Σ, L(A) is empty } is decidable.

A

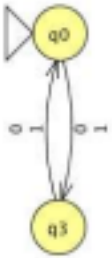
L(A)=Gy =b

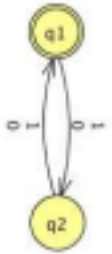
Proving decidability

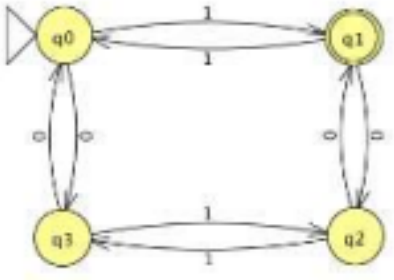
Claim: E_{DFA} is decidable

Proof: WTS that { <A> | A is a DFA over Σ, L(A) is empty } is decidable.

A B







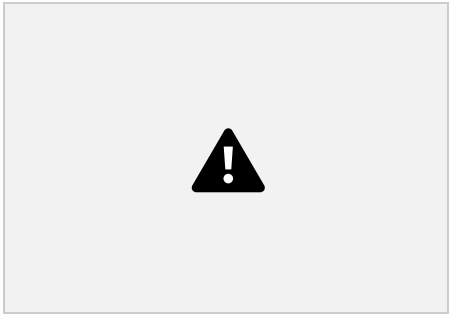
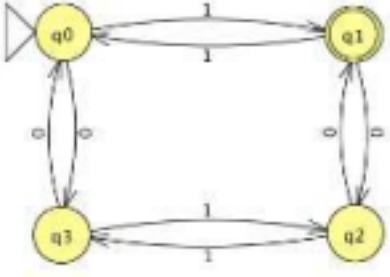
LJA) ((3) = (1 11 . .-. -]

should accept should reject

Proving decidability

Claim: E_{DFA} is decidable

Proof: WTS that { <A> | A is a DFA over Σ, L(A) is empty } is decidable.

e.g.<  > is in E_{DFA} ; < 

> is not in E_{DFA}

E_{DFA} should accept should reject

Proving decidability

Claim: E_{DFA} is decidable

Proof: WTS that { <A> | A is a DFA over Σ, L(A) is empty } is decidable.

Idea: give high-level description

Step 1: construction

*What condition distinguishes between DFA that accept *some* string and those that don't accept *any*?*

Proving decidability

Claim: E_{DFA} is decidable

Proof: WTS that { <A> | A is a DFA over Σ, L(A) is empty } is decidable.

Idea: give high-level

description Step 1: construction

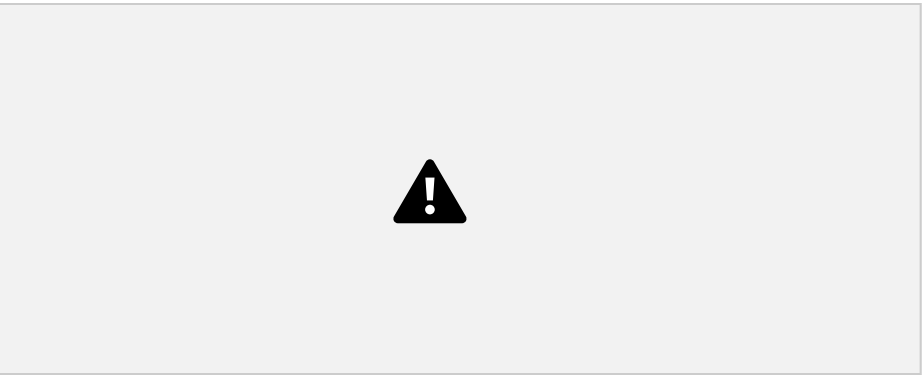


diagram to look ↑t
for path from state-state to an accepting state

Breadth first search in transition

*What condition distinguishes between DFA that accept *some* string and those that don't accept *any*?*

Proving decidability

Claim: E_{DFA} is decidable

Proof: WTS that { <A> | A is a DFA over Σ, L(A) is empty }
is decidable. Idea: give high-level description

Step 1: construction "On input <A>:

-> $351 2] Fin]

Define TM M_2 by: M_2 =

1. Check whether A is a valid encoding of a DFA; if not, reject.
2. Mark the start state of A.
3. Repeat until no new states get marked:
   i. Loop over states of A and mark any unmarked state that has an **incoming** edge from a marked state.

4. If no accept state of A is marked, *accept*; otherwise, *reject.*

Proving decidability

Step 1: construction

Define TM M_2 by: M_2 = "On input <A>:

1. Check whether A is a valid encoding of a DFA; if not, reject.
2. Mark the state state of A.
3. Repeat until no new states get marked:
   i. Loop over states of A and mark any unmarked state that has an **incoming** edge from a marked state.
4. If no accept state of A is marked, *accept*; otherwise, *reject.*

Step 2: correctness proof

WTS (1) $L(M_2) = E_{DFA}$ and (2) M_2 is a decider.

& R ,W) / R is regular exp ,w not generated by RY

Techniques *Sipser4.1* Re-NFA-DFA

- **Subroutines**: can use decision procedures of decidable problems as subroutines in other algorithms
  - A_{DFA}
  - E_{DFA}
  - EQ_{DFA}
- **Constructions**: can use algorithms for constructions as subroutines in other algorithms

- Converting DFA to DFA recognizing complement (or Kleene star).
- Converting two DFA/NFA to one recognizing union (or intersection, concatenation).
- Converting NFA to equivalent DFA.
- Converting regular expression to equivalent NFA.
- Converting DFA to equivalent regular expression.

Computational problems

A computational problem is **decidable** iff the language encoding the problem instances is decidable

Computational problems -

Sample computational problems and their encodings:

- A_{DFA} "Check whether a string is accepted by a DFA."
  { <B,w> | B is a DFA over Σ, w in Σ*, and w is in L(B)

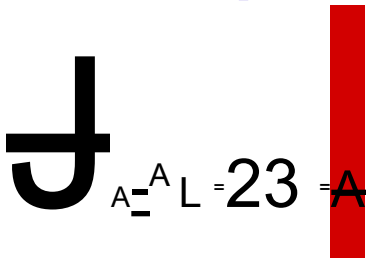
} -

- E_{DFA} "Check whether the language of a DFA is empty."
  { <A> | A is a DFA over Σ, L(A) is empty
  }

- EQ_{DFA} "Check whether the languages of two DFAs are equal."
  { <A, B> | A and B are DFA over Σ, L(A) = L(B)}

FACT: all of these problems are decidable!

Computational problems



Gi

Sample computational problems and their encodings:

Decidable

• A_{PDA} "Check whether a string ((a) D is accepted by a PDA."
{ <B,w> | B is a PDA over Σ, w in Σ*, and w is in L(B) }

• E_{PDA} "Check whether the language

of a PDA is empty." Decidable

PDA ?

CFG CFL EPDA

{ <A> | A is a PDA over Σ, L(A) is empty }

- EQ_{PDA} "Check whether the languages of two PDAs are equal." { <A, B> | A and B are PDA over Σ, L(A) = L(B)}

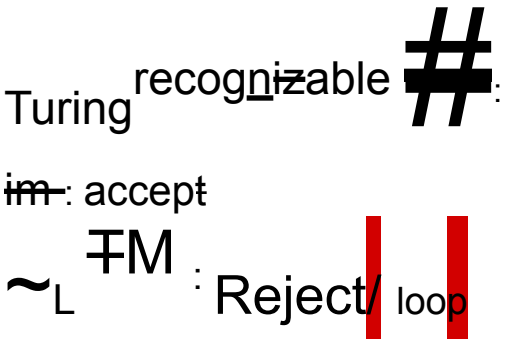
undecidable, there is no general algorithm that can always decide whether **FACT: some**

of these problems are decidable, and some are not!

Computational problems

Sample computational problems and their encodings:

- A_{TM} "Check whether a string is accepted by a TM."



{ <B,w> | B is a TM over Σ, w in Σ*, and w is in L(B) }

- E_{TM} "Check whether the language of a TM is empty." { <A> | A is a TM over Σ, L(A) is empty }

- EQ_{TM} "Check whether the languages of two TMs are equal."

{ <A, B> | A and B are TM over Σ, L(A) = L(B)}

FACT: all of these problems are undecidable!

Undecidable?

- There are many ways to prove that a problem ***is*** decidable.
- How do we find (and prove) that a problem ***is not*** decidable?

Aim: undecidable

~~Haltty~~ Halting problem

Counting arguments

*

Given a TM M and input w, will M eventually halt?

& {[0,13* &ww starts with by

Before we proved the Pumping Lemma …

8 ,1 ,101

We proved there was a set that was not regular because

Uncountable

All Regular Sets

Countable

TM Can be listed. Each recognizable lang Come from at least one TM So

recognizable

Counting arguments

lang can be listed. Therefore they are countable.

Turing recognizable sets

Countable

Uncountable

All

Why is the set of Turing-recognizable languages **countable**? A. It's

equal to the set of all TMs, which we showed is countable. B. It's a subset of the set of all TMs, which we showed is countable. ~

C. Each Turing-recognizable language is associated with a TM, so there can be no more Turing-recognizable languages than TMs. D. More than one of the above.

E. I don't know.

Satisfied?

- Maybe not …

- What's a specific example of a language that is not Turing-recognizable? or not Turing-decidable?

- *Idea: consider a set that, were it to be Turing-decidable, would have to "talk" about itself, and contradict itself!*

A_{TM} Decider

-

Recall A_{DFA} = {<B,w> | B is a DFA and w is in L(B) } Decider for this set simulates arbitrary DFA

~~TM~~ Simulating DFA mush Simpler !

A_{TM} = {<M,w> | M is a TM and w is in L(M) }

Decider for this set simulates arbitrary TMs

??? ↓

Decidery? Aim ' Simulating a TM

If Atm Simulate itself , it can break

A_{TM}

A_{TM} = {<M,w> | M is a TM and w is in L(M)

} Define the TM N = "On input <M,w>:

1. Simulate M on w.

What if m loops forever in W] Then N

2. If M accepts, accept. If M rejects, reject."

Atm is Turing recognizable

will loop forever

N recognizes ATM

A_{TM}

A_{TM} = {<M,w> | M is a TM and w is in L(M) }

Define the TM N = "On input <M,w>: 1. Simulate M on w.

2. If M accepts, accept. If M rejects, reject."

What is L(N)?

A_{TM}

A_{TM} = {<M,w> | M is a TM and w is in L(M) }

Define the TM N = "On input

<M,w>: 1. Simulate M on w.

2. If M accepts, accept. If M rejects, reject."

Does N decide A_{TM}?

A_{TM}

A_{TM} = {<M,w> | M is a TM and w is in L(M) }

Define the TM N = "On input <M,w>: 1. Simulate M on w.

2. If M accepts, accept. If M rejects, reject."

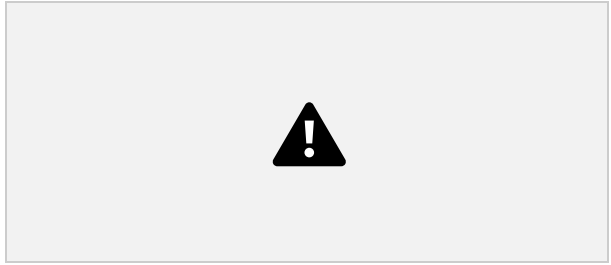
Conclusion: A_{TM} is Turing-recognizable. **Is it decidable?**

Diagonalization proof: A_{TM} not decidable

Sipser 4.11 Assume, towards a contradiction, that it is.

Call $\mathbf{M_{ATM}}$ the decider for A_{TM}:

$\mathbf{M_{ATM}} \neq$ N Not real, it is a fantasy Machine WHW



For every TM M and every string w,

Assume alt Frdet

- Computation of M_{ATM} on <M,w> halts and accepts if w is in L(M).
- Computation of M_{ATM} on <M,w> halts and rejects if w is not in L(M).

Diagonalization proof: A_{TM} not decidable

Sipser 4.11 Assume, towards a contradiction, that M_{ATM} decides

A_{TM}

Define the TM D = "On input <M>:

1. Run M_{ATM} on <M, <M>>. (M)

Ask Whether Machine M accept its own encoding

L V

2. If M_{ATM} accepts, reject; if M_{ATM} rejects, accept."

D is a flipping machine

D does the opposite of what M does on (M)

Diagonalization proof: A_{TM} not decidable

Sipser 4.11 Assume, towards a contradiction, that M_{ATM} decides A_{TM}

Define the TM D = "On input <M>:

1. Run M_{ATM} on <M, <M>>.

2. If M_{ATM} accepts, reject; if M_{ATM} rejects, accept."

Consider **running D on input <D>**. Because D is a decider: **either computation halts and accepts** …

 or computation halts and rejects …

Diagonalization proof: A_{TM} not decidable

Sipser4.11 Assume, towards a contradiction, that M_{ATM} decides A_{TM}

Define the TM D = "On input <M>:

1. Run M_{ATM} on <M, <M>>.

2. If M_{ATM} accepts, reject; if M_{ATM} rejects, accept."

- If D(<D>) accepts, then M_{ATM}(<D>,<D>) rejects, which means D(<D>) should have rejected
- If D(<D>) rejects, then M_{ATM}(<D>,<D>) accepts, which means D(<D>) should have accepted
- Either way, we reached a **contradiction**

Recall:

D (<M>] runs Main) <M,(M))

Then flips the answer.

Now put D itself as input:

D((D))

So D asks -Does D accept (D)

Then it does the opposite

Now there are two cases :

1) Suppose D((D) accepts

* If D accepts , that means Main must have rejected , because flips the answer

But MATM rejecting (D) means

& does not accept <D)

Case 2 : Suppose D(D)) rejects

If D rejects, Main must accept

But Math accepting <D,<D)) means D accepts (D)

Contradiction !

the original assumption was false

So, Atm is undecidable. and recognizable

Sipser 4.11 Assume, towards a contradiction, that M_{ATM} decides A_{TM}

Define the TM D = "On input <M>:

1. Run M_{ATM} on <M, <M>>.

2. If M_{ATM} accepts, reject; if M_{ATM} rejects, accept."

Consider **running D on input <D>**. Because D is a

decider: **either computation halts and accepts …**

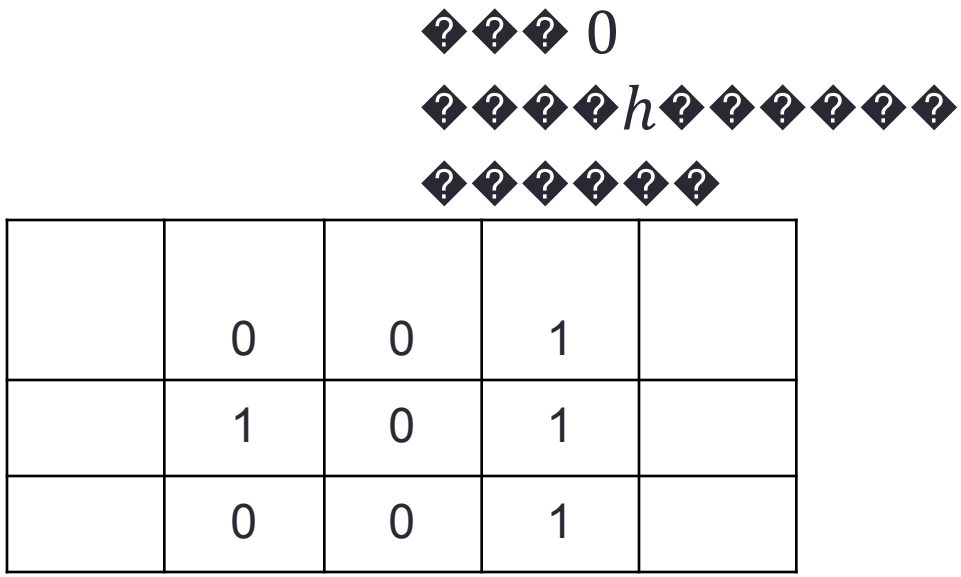
 or computation halts and rejects …

Why is this called diagonalization?

- Let M_1, M_2, … be an enumeration of **all TMs** •

Let w_1, w_2, … be an enumeration of **all inputs**

- Construct table: ▯▯ ▯▯,▯▯ = ▯▯ ▯▯▯▯



Why is this called diagonalization?

	0	0	1	
	1	0	1	
	0	0	1	

- Rows of table describe **all recognizable languages**
- Any other row (=language) cannot be recognizable
- We will construct such a language

Why is this called diagonalization?

- Next idea: specialize only to inputs $\langle M_i \rangle$

	0	0	1	
	1	0	1	
	0	0	1	

- Suffices to construct row (=language) that differs from any row on some of these inputs

Why is this called diagonalization?

- Consider the **diagonal**: running M_i on $\langle M_i \rangle$; then **flip it**

	0	0	1	
	1	**0**	1	
	0	0	**1**	

- L = {<M>: M doesn't accept <M>}
- L cannot be recognized by any M_i – they disagree on input $<M_i>$
- Since we consider all TMs, L must be **un-recognizable**